program3

November 6, 2026

1 CSC3310 Program #3: Dynamic Programming

1.1 Objectives

- Design a dynamic programming algorithm for a given computational problem
- Justify the correctness of an algorithm
- Perform asymptotic complexity analysis of the run time
- Design and execute benchmarks for an algorithm

1.2 Overview

For this assignment, you will apply dynamic programming to create an algorithm that solves the maximal substring problem.

1.3 The Problem

The problem that you will be solving is called the **MAXIMALSUBSTRING** problem.

1.3.1 MAXIMALSUBSTRING

- **Input:** A string a of n characters, and a string b of m characters
- **Output:** A string s such that s is a substring of both a and b, and there is no string s' where $|s'| > |s|$ and s' is a substring of both a and b

For example, if we are given the strings `"orange"` and `"banana"`, we would return `"an"`; for `"banana"` and `"apple"`, we return `"a"`; and for `"apple"` and `"pineapple"`, we return `"apple"`. For `"pineapple"` and `"orange"`, we could return either `"e"` or `"a"`.

There are $O(n^2)$ possible substrings in a and (similarly) $O(m^2)$ in b. We can avoid some complexity by only checking substrings of the same length, but a brute-force algorithm will still check $O(n^2m)$ pairs.

I strongly encourage you to use character operations rather than string operations—they will be easier to index usefully, and are no less efficient.

> **N.B.** String operations are not constant time. Comparing two strings takes $O(k)$ time, where k is the string length. Similarly, concatenation $x + y$ takes $O(k_y)$ time (some implementations require $k_x + k_y$; we will assume ours is not one such). Slicing (getting a substring) needs time proportional to the length of the substring.

Your goal is to create a faster algorithm using a dynamic-programming strategy. To do this, you will first implement a backtracking (brute-force) strategy, then memoize it lazily while keeping the structure of the backtracking algorithm, then rebuild the dynamic-programming algorithm to eagerly fill the appropriate array.

1.4 Working Through the Process

1. **Start by implementing the backtracking algorithm.** For this, you will need to design a solution that tests a series of decisions. The algorithm should use recursion to test different potential choices.
   - *Hint:* Consider substrings that end at a particular pair of characters (one in each string). Consider sequences of characters, rather than `String` objects.
2. **Once you have the backtracking algorithm working correctly, memoize it.** Find a way to index results and store them in an array, so that they can be used later in place of recursive calls.
   - *Hint:* An index can be used to represent the character at that index in one of the strings, presuming you have results tied to a specific character.
3. **Finally, rewrite the recursive function to eagerly fill the array.** Here it is useful to consider which elements need to be filled before which others, and to then structure your loops accordingly.

1.5 Deliverables

Complete a lab report containing the following: * A paragraph describing the approach you used to solve the problem. Provide at least 2 illustrations that explain the approach. * High-level pseudocode for each of the three algorithms, each of which solves the computational problem for any input. * An explanation and justification for why each of your algorithms is correct (1–3 paragraphs apiece). * A table of your test cases, the answers you expect, and the answers returned by your implementation of each algorithm. * An analysis of the asymptotic runtime of each algorithm, using techniques from this class. * A table and graph from benchmarking different lists with different sizes and values of m and n. * An analysis, comparing the runtimes for each algorithm against both your expectations and each other.
* *Note:* If the benchmarks do not support your theoretically-derived run time and/or do not provide evidence that the run time of your algorithm grows more slowly than the brute-force approach, this may indicate a flaw in your implementation. * An appendix containing all of your source code and test cases.

Submit your report to Canvas in PDF format.
In addition to the report, you should submit a zip archive with your source code.

1.6 Rubric

Criterion	Weight	Full Credit	Partial Credit	No Credit
In-class participation	5%	Active engagement in class activities.	Partial engagement.	No participation.
Lab report writing and presentation quality	10%	Professional, well-organized, and clear writing.	Minor issues in formatting, organization, or clarity.	Poorly organized or written.
Solution Approach Description	10%	Approach is well-described, with all necessary details to implement and/or to explain to someone else. Includes at least 2 illustrations.	Approach is decently described with most details necessary to implement. Explanation may be unclear in places.	The description is insufficient to implement or explain to others.
Algorithm, as described in Pseudocode	15%	Algorithms are correct for all allowed inputs. Memoized and eager algorithms follow from original backtracking algorithm.	Algorithm is correct for most inputs. Algorithms are loosely related.	Algorithm is incorrect for many inputs. Not all algorithms present.
Justification of Correctness	10%	Uses techniques described in class to provide a solid and convincing argument that the algorithm is correct.	Provides a somewhat convincing argument that the algorithm is correct.	Argument contains one or more serious flaws.
Asymptotic run-time analysis	10%	Analysis is correct for the provided pseudocode.	Analysis contains minor flaws.	Analysis is significantly flawed.
Algorithm Implementations	10%	Each implementation is both faithful to the pseudocode and correct.	Implementations are mostly faithful to the pseudocode and/or correct for most inputs.	An implementation is not faithful to the pseudocode or not correct for common inputs.

Criterion	Weight	Full Credit	Partial Credit	No Credit
Test Cases	10%	Test cases consider a range of problem sizes, complexities, classes, and edge cases.	Limited number of test cases only testing obvious or simple cases.	Only uses the provided assignment examples.
Benchmarking	15%	Benchmark experiments were set up, implemented, and interpreted correctly.	Benchmark experiments, implementations, or results are mostly correct. Interpretation was cursory.	Benchmark experiments, implementations, or results are flawed. No interpretation was provided.
Submitted correctly	5%	PDF report and zip archive submitted correctly to Canvas.	Missing one component or submitted in incorrect format.	Not submitted properly.

1.7 Deliverables

Complete a lab report containing the following: * A paragraph describing the approach you used to solve the problem. Provide at least 2 illustrations that explain the approach. * High-level pseudocode for each of the three algorithms, each of which solves the computational problem for any input. * An explanation and justification for why each of your algorithms is correct (1–3 paragraphs apiece). * A table of your test cases, the answers you expect, and the answers returned by your implementation of each algorithm. * An analysis of the asymptotic runtime of each algorithm, using techniques from this class. * A table and graph from benchmarking different lists with different sizes and values of m and n. * An analysis, comparing the runtimes for each algorithm against both your expectations and each other.
* *Note:* If the benchmarks do not support your theoretically-derived run time and/or do not provide evidence that the run time of your algorithm grows more slowly than the brute-force approach, this may indicate a flaw in your implementation. * An appendix containing all of your source code and test cases.

1.8 Approach

The core logic of our algorithm checks the common suffix length for all pairs of end indices for each string, then keeps track of the longest common suffix and returns that substring.

This works because every substring of a and b must terminates at some pair of indices (i, j). Our algorithm computes the length of the longest common suffix ending at every pair (i, j). Because every valid common substring corresponds to a common suffix for some pair of endpoints, tracking the maximum value across all (i, j) guarantees that the globally longest common substring is identified without checking redundant slices.

We start with a backtracking approach that uses a recursive sub-algorithm to return the common

suffix length for some pair of indices (i, j). Then we loop through all pairs of indices and use that recursive function to calculate the common suffix for each pair while keeping track of the maximum.

Then we convert the algorithm into a dynamic-programming approach by using a 2D memo $(n \times m)$ to keep track of the common suffix length for all pairs of end indices (i, j) to avoid doing redundant computations for the same pairs of indices.

Finally, we convert the algorithm into an eager approach that builds the memo bottom up without using recursive calls since each element only depends on the row and column before it.

We can also further optimize this by only keeping a single array of length $min(n, m)$ instead of a $n \times m$ table. In order to do this we swap a and b if needed to make sure b is the shorter string, then we iterate i from 1 to n and j from m to 1 to ensure that we don't overwrite the value from the previous row while only using a single array. Since the algorithm is symmetric, swapping a and b doesn't change the result.

1.8.1 Examples

Using two memo tables for different worked examples is an effective way to satisfy the requirement for "at least 2 illustrations that explain the approach". Comparing two distinct cases clearly highlights how the dynamic programming algorithm behaves under different string configurations.

Here is how you can present both tables in your report:

Illustration 1 (a = "cat", b = "bat") This example demonstrates how contiguous matches extend diagonally to build a substring longer than a single character.

	ε	b $(j = 0)$	a $(j = 1)$	t $(j = 2)$
ε	0	0	0	0
c $(i = 0)$	0	0	0	0
a $(i = 1)$	0	0	**1** (`"a"`)	0
t $(i = 2)$	0	0	0	**2** (`"at"`)

- **Maximal Length:** 2 at $(i = 2, j = 2)$
- **Resulting Substring:** $a[2 - 2 + 1 \dots 2] = a[1 \dots 2] \implies$ "at"

This table shows memo entries for inputs "cat" and "bat". The matching characters 'a' and 't' align consecutively along the diagonal, propagating the match length from 1 to 2 to identify the maximal common substring "at".

Illustration 2 (a = "pot", b = "top") This example illustrates how even though the two strings share multiple identical characters, none appear consecutively in the same relative diagonal order, causing values to reset to 0.

	ε	t ($j = 0$)	o ($j = 1$)	p ($j = 2$)
ε	0	0	0	0
p ($i = 0$)	0	0	0	**1** ("p")
o ($i = 1$)	0	0	**1** ("o")	0
t ($i = 2$)	0	**1** ("t")	0	0

- **Maximal Length:** 1 (tied at $(0, 2)$, $(1, 1)$, and $(2, 0)$)
- **Resulting Substring:** Any of the single characters: "p", "o", or "t"

This table shows memo entries for inputs "pot" and "top". Although 'p', 'o', and 't' all match at isolated positions, their off-diagonal alignment prevents values from accumulating beyond length 1, demonstrating how mismatches reset substring growth.

1.9 Pseudocode

1.9.1 Backtracking

```
Algorithm MAXIMAL-SUBSTRING-BACKTRACKING(a, b):
    n = length(a)
    m = length(b)

    if n == 0 or m == 0:
        return ""

    Function COMMON-SUFFIX-LEN(i, j):
        if i < 0 or j < 0:
            return 0

        if a[i] == b[j]:
            return 1 + COMMON-SUFFIX-LEN(i - 1, j - 1)
        else:
            return 0

    max_len = 0
    end_idx_a = -1

    for i from 0 to n - 1:
        for j from 0 to m - 1:
            curr_len = COMMON-SUFFIX-LEN(i, j)
            if curr_len > max_len:
                max_len = curr_len
                end_idx_a = i

    if max_len == 0:
        return ""

    start_idx_a = end_idx_a - max_len + 1
    return a[start_idx_a : end_idx_a + 1]
```

Justification This algorithm finds the longest common substring (LCS) by evaluating every candidate pair of ending indices where a common substring could terminate. Any non-empty common substring must end at some character index i in a ($0 \leq i < n$) and some index j in b ($0 \leq j < m$). Therefore, finding the LCS reduces to finding the pair (i, j) that maximizes the length of their shared common suffix, which is computed by `COMMON-SUFFIX-LEN(i, j)`.

Proof of Correctness for `COMMON-SUFFIX-LEN(i, j)` by Induction Let $L(i, j)$ be the true length of the longest common suffix of prefixes $a[0 \ldots i]$ and $b[0 \ldots j]$. We prove by mathematical induction on $k = \min(i, j) + 1 \geq 0$ that `COMMON-SUFFIX-LEN(i, j)` correctly returns $L(i, j)$ for all $i, j \in \mathbb{Z}$:

- **Base Case ($k = 0$):** If $k = 0$, then at least one index is negative ($i < 0$ or $j < 0$). In this case, at least one prefix is empty, so no common suffix can exist ($L(i, j) = 0$). The function checks `if i < 0 or j < 0` and returns `0`, which is correct.
- **Inductive Step ($k \geq 1$):** Assume the induction hypothesis holds: for all pairs (i', j') where $\min(i', j') + 1 < k$, `COMMON-SUFFIX-LEN(i', j')` correctly returns $L(i', j')$. Now consider a pair (i, j) with $\min(i, j) + 1 = k$:
  - **Case 1: $a[i] \neq b[j]$.** The characters at the terminal positions do not match. Since any common suffix must match at its final character, the length of the common suffix ending at (i, j) is strictly 0 ($L(i, j) = 0$). The function enters the `else` branch and returns `0`, which is correct.
  - **Case 2: $a[i] == b[j]$.** The characters at $a[i]$ and $b[j]$ match, contributing 1 character to the suffix. Any additional matching characters immediately preceding them must form a common suffix between $a[0 \ldots i-1]$ and $b[0 \ldots j-1]$. Thus, by optimal substructure:

    $$L(i, j) = 1 + L(i-1, j-1)$$

    Because $\min(i-1, j-1) + 1 = k - 1 < k$, by the induction hypothesis the recursive call `COMMON-SUFFIX-LEN(i - 1, j - 1)` correctly returns $L(i-1, j-1)$. Therefore, the function returns $1 + L(i-1, j-1) = L(i, j)$, which is correct.

So, by the principle of mathematical induction, `COMMON-SUFFIX-LEN(i, j)` correctly returns the longest common suffix length for all valid indices.

Global Optimality & Reconstruction The nested loops iterate over all possible pairs $(i, j) \in [0, n-1] \times [0, m-1]$, ensuring no candidate ending position is missed. Whenever `curr_len > max_len`, the algorithm updates `max_len` and captures the ending position in a (`end_idx_a = i`), and since the search space is finite and fully traversed, `max_len` is guaranteed to be the global maximum length over all common substrings.

Finally, since the maximal substring ends at `end_idx_a` and has length `max_len`, its starting index is can be determined by start_idx_a = end_idx_a − max_len + 1. Then slicing a[start_idx_a : end_idx_a + 1] correctly recovers the longest common substring.

1.9.2 Memoized

```
Algorithm MAXIMAL-SUBSTRING-MEMOIZED(a, b):
    n = length(a)
    m = length(b)
```

```
if n == 0 or m == 0:
    return ""

memo = [[-1 for col in range(m)] for row in range(n)]

Function COMMON-SUFFIX-LEN(i, j):
    if i < 0 or j < 0:
        return 0

    if memo[i][j] != -1:
        return memo[i][j]

    if a[i] == b[j]:
        memo[i][j] = 1 + COMMON-SUFFIX-LEN(i - 1, j - 1)
    else:
        memo[i][j] = 0

    return memo[i][j]

max_len = 0
end_idx_a = -1

for i from 0 to n - 1:
    for j from 0 to m - 1:
        curr_len = COMMON-SUFFIX-LEN(i, j)
        if curr_len > max_len:
            max_len = curr_len
            end_idx_a = i

if max_len == 0:
    return ""

start_idx_a = end_idx_a - max_len + 1
return a[start_idx_a : end_idx_a + 1]
```

Justification This version improves the previous algorithm using memoization to eliminate redundant work. In the original backtracking version, calling `COMMON-SUFFIX-LEN(i, j)` repeatedly traversed the same diagonal pairs `(i - 1, j - 1), (i - 2, j - 2)...` from scratch across different outer-loop iterations, resulting in wasted computations.

Here, a 2D table `memo` of size $n \times m$ is initialized to `-1` to store previously solved subproblems. When `COMMON-SUFFIX-LEN(i, j)` is called, it first checks if `memo[i][j]` already holds an answer; if so, it returns that value instantly in $O(1)$ time rather than recursing. That way it only calculate the suffix length and saves the result into `memo[i][j]` when visiting an uncomputed state.

As a result, each pair `(i, j)` is computed at most once. This drops the worst-case time complexity to $O(n \cdot m)$, requiring $O(n \cdot m)$ additional space for the cache.

1.9.3 Eager

```
Algorithm MAXIMAL-SUBSTRING-EAGER(a, b):
    n = length(a)
    m = length(b)

    if n == 0 or m == 0:
        return ""

    memo = [[0 for col in range(m + 1)] for row in range(n + 1)]

    max_len = 0
    end_idx_a = -1

    for i from 1 to n:
        for j from 1 to m:
            if a[i - 1] == b[j - 1]:
                memo[i][j] = 1 + memo[i - 1][j - 1]
                if memo[i][j] > max_len:
                    max_len = memo[i][j]
                    end_idx_a = i - 1
            else:
                memo[i][j] = 0

    if max_len == 0:
        return ""

    start_idx_a = end_idx_a - max_len + 1
    return a[start_idx_a : end_idx_a + 1]
```

Justification This version converts the top-down memoized approach into bottom-up eager dynamic programming. Instead of using recursion and checking a cache on-demand, it iteratively fills out an $(n+1) \times (m+1)$ table from smallest subproblems to largest.

By adding an extra row and column of zeros at index 0 to represent empty strings, it eliminates bounds checking and base-case logic. Each cell `memo[i][j]` simply looks up the already-computed diagonal neighbor `memo[i - 1][j - 1]`. If `a[i - 1] == b[j - 1]`, it adds 1 to that value; otherwise, it sets it to 0.

The maximum length and ending position are updated inside the same loop. This keeps the $O(n \cdot m)$ time and space complexity of the memoized version while completely removing recursive function call overhead.

1.9.4 Eager (Optimized)

```
Algorithm MAXIMAL-SUBSTRING-EAGER-OPTIMIZED(a, b):
    if length(a) < length(b):
        c = a
        a = b
```

```
    b = c

n = length(a)
m = length(b)

if n == 0 or m == 0:
    return ""

memo = [0 for row in range(m + 1)]

max_len = 0
end_idx_a = -1

for i from 1 to n:
    for j from m to 1:
        if a[i - 1] == b[j - 1]:
            memo[j] = 1 + memo[j - 1]
            if memo[j] > max_len:
                max_len = memo[j]
                end_idx_a = i - 1
        else:
            memo[j] = 0

if max_len == 0:
    return ""

start_idx_a = end_idx_a - max_len + 1
return a[start_idx_a : end_idx_a + 1]
```

Justification This version optimizes the bottom-up approach by dramatically reducing memory usage. Since the current row only requires values from the previous row's diagonal entry, it replaces the 2D table with a single array.

To achieve this in a single 1D array of size $m+1$, the inner loop iterates **backward** (`for j from m to 1`). Walking in reverse ensures that when evaluating `memo[j] = 1 + memo[j - 1]`, the value at `memo[j - 1]` still belongs to the *previous* row (`i - 1`) rather than having already been overwritten by the current row.

Additionally, swapping `a` and `b` at the start guarantees that string `b` is always the shorter one ($m \leq n$). This shrinks the auxiliary space complexity from $O(n \cdot m)$ down to $O(\min(n, m))$, while maintaining the optimal $O(n \cdot m)$ runtime.

1.10 Tests Cases

We'll use the following test cases to evaluate edge cases and verify that all algorithms work correctly.

Test Case	string a	string b	Expected Output
Standard	"orange"	"banana"	"an"

Test Case	string a	string b	Expected Output
Single Character	“banana”	“apple”	“a”
Full String	“apple”	“pineapple”	“apple”
No Match	“abc”	“xyz”	“ ”
Identical	“hello”	“hello”	“hello”
Empty String	“ ”	“apple”	“ ”

[1]:
```python
# Algorithm Implementations

def maximal_substring_backtracking(a: str, b: str) -> str:
    n, m = len(a), len(b)
    if n == 0 or m == 0:
        return ""

    def common_suffix_len(i: int, j: int) -> int:
        if i < 0 or j < 0:
            return 0
        if a[i] == b[j]:
            return 1 + common_suffix_len(i - 1, j - 1)
        return 0

    max_len = 0
    end_idx_a = -1

    for i in range(n):
        for j in range(m):
            curr_len = common_suffix_len(i, j)
            if curr_len > max_len:
                max_len = curr_len
                end_idx_a = i

    if max_len == 0:
        return ""
    start_idx_a = end_idx_a - max_len + 1
    return a[start_idx_a : end_idx_a + 1]


def maximal_substring_memoized(a: str, b: str) -> str:
    n, m = len(a), len(b)
    if n == 0 or m == 0:
        return ""

    memo = [[-1 for _ in range(m)] for _ in range(n)]

    def common_suffix_len(i: int, j: int) -> int:
        if i < 0 or j < 0:
```

```
            return 0
        if memo[i][j] != -1:
            return memo[i][j]

        if a[i] == b[j]:
            memo[i][j] = 1 + common_suffix_len(i - 1, j - 1)
        else:
            memo[i][j] = 0
        return memo[i][j]

    max_len = 0
    end_idx_a = -1

    for i in range(n):
        for j in range(m):
            curr_len = common_suffix_len(i, j)
            if curr_len > max_len:
                max_len = curr_len
                end_idx_a = i

    if max_len == 0:
        return ""
    start_idx_a = end_idx_a - max_len + 1
    return a[start_idx_a : end_idx_a + 1]


def maximal_substring_eager(a: str, b: str) -> str:
    n, m = len(a), len(b)
    if n == 0 or m == 0:
        return ""

    memo = [[0 for _ in range(m + 1)] for _ in range(n + 1)]
    max_len = 0
    end_idx_a = -1

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                memo[i][j] = 1 + memo[i - 1][j - 1]
                if memo[i][j] > max_len:
                    max_len = memo[i][j]
                    end_idx_a = i - 1
            else:
                memo[i][j] = 0

    if max_len == 0:
        return ""
```

```
    start_idx_a = end_idx_a - max_len + 1
    return a[start_idx_a : end_idx_a + 1]


def maximal_substring_eager_optimized(a: str, b: str) -> str:
    if len(a) < len(b):
        a, b = b, a

    n, m = len(a), len(b)
    if n == 0 or m == 0:
        return ""

    memo = [0 for _ in range(m + 1)]
    max_len = 0
    end_idx_a = -1

    for i in range(1, n + 1):
        for j in range(m, 0, -1):
            if a[i - 1] == b[j - 1]:
                memo[j] = 1 + memo[j - 1]
                if memo[j] > max_len:
                    max_len = memo[j]
                    end_idx_a = i - 1
            else:
                memo[j] = 0

    if max_len == 0:
        return ""
    start_idx_a = end_idx_a - max_len + 1
    return a[start_idx_a : end_idx_a + 1]




# Tests

test_cases = [
    {"name": "Standard",         "a": "orange", "b": "banana",    "expected":␣
 ↪"an"},
    {"name": "Single Character", "a": "banana", "b": "apple",     "expected":␣
 ↪"a"},
    {"name": "Full String",      "a": "apple",  "b": "pineapple", "expected":␣
 ↪"apple"},
    {"name": "No Match",         "a": "abc",    "b": "xyz",       "expected":␣
 ↪""},
    {"name": "Identical",        "a": "hello",  "b": "hello",     "expected":␣
 ↪"hello"},
```

```
    {"name": "Empty String",     "a": "",       "b": "apple",     "expected":␣
  ↪""},
]

algorithms = {
    "Backtracking": maximal_substring_backtracking,
    "Memoized": maximal_substring_memoized,
    "Eager": maximal_substring_eager,
    "Eager Optimized": maximal_substring_eager_optimized,
}

print(f"{'Test Case':<18} | {'Algorithm':<16} | {'Expected':<10} | {'Output':
  ↪<10} | {'Status'}")
print("-" * 70)

all_passed = True
for tc in test_cases:
    for algo_name, algo_fn in algorithms.items():
        actual = algo_fn(tc["a"], tc["b"])
        passed = (actual == tc["expected"])
        status = "PASSED" if passed else "FAILED"
        if not passed:
            all_passed = False
        print(f"{tc['name']:<18} | {algo_name:<16} | {repr(tc['expected']):<10}␣
  ↪| {repr(actual):<10} | {status}")

print("-" * 70)
print("All tests passed!" if all_passed else "Some tests failed!")
```

```
Test Case          | Algorithm        | Expected   | Output     | Status
----------------------------------------------------------------------
Standard           | Backtracking     | 'an'       | 'an'       | PASSED
Standard           | Memoized         | 'an'       | 'an'       | PASSED
Standard           | Eager            | 'an'       | 'an'       | PASSED
Standard           | Eager Optimized  | 'an'       | 'an'       | PASSED
Single Character   | Backtracking     | 'a'        | 'a'        | PASSED
Single Character   | Memoized         | 'a'        | 'a'        | PASSED
Single Character   | Eager            | 'a'        | 'a'        | PASSED
Single Character   | Eager Optimized  | 'a'        | 'a'        | PASSED
Full String        | Backtracking     | 'apple'    | 'apple'    | PASSED
Full String        | Memoized         | 'apple'    | 'apple'    | PASSED
Full String        | Eager            | 'apple'    | 'apple'    | PASSED
Full String        | Eager Optimized  | 'apple'    | 'apple'    | PASSED
No Match           | Backtracking     | ''         | ''         | PASSED
No Match           | Memoized         | ''         | ''         | PASSED
No Match           | Eager            | ''         | ''         | PASSED
No Match           | Eager Optimized  | ''         | ''         | PASSED
```

```
Identical          | Backtracking     | 'hello'    | 'hello'    | PASSED
Identical          | Memoized         | 'hello'    | 'hello'    | PASSED
Identical          | Eager            | 'hello'    | 'hello'    | PASSED
Identical          | Eager Optimized  | 'hello'    | 'hello'    | PASSED
Empty String       | Backtracking     | ''         | ''         | PASSED
Empty String       | Memoized         | ''         | ''         | PASSED
Empty String       | Eager            | ''         | ''         | PASSED
Empty String       | Eager Optimized  | ''         | ''         | PASSED
------------------------------------------------------------------------
All tests passed!
```

1.11 Asymptotic Runtime

1.11.1 Backtracking

For COMMON-SUFFIX-LEN(n,m) (note, n in this case is whichever is smaller, n or m)

$R(n) = R(n-1)+2R(n) = an+ban+b = a(n-1)+b+20 = -a+2a = 2R(n) = 2n+b$with base case $R(0) = 1 \implies b$

Proof for $\Theta(n)$:

$$c_1 n \leq 2n + 1 \leq c_2 n$$

Left Side: (pick $c_1 = 2$)

$$2n \leq 2n + 10 \leq 1 \therefore \text{True for all } n \geq 1$$

Right Side: (pick $c_2 = 3$)

$$2n + 1 \leq 3n1 \leq n \therefore \text{True for all } n \geq 1$$

Conclusion:

$$\therefore R(n)\Theta(n)$$

For the overall time complexity: (im assuming n is the bigger one)

$$T(n, m) = nm(R(m) + 3) + 5T(n, m) = nm(m + 3) + 5T(n, m) = nm^2 + 3nm + 5$$

Proof for $\Theta(nm^2)$:

$$c_1 nm^2 \leq nm^2 + 3nm + 5 \leq c_2 nm^2$$

Left Side: (pick $c_1 = 1$)

$$1nm^2 \leq nm^2 + 3nm + 50 \leq 3nm + 5 \therefore \text{True for all } n, m \geq 1$$

Right Side: (pick $c_2 = 9$)

$$nm^2 + 3nm + 5 \leq nm^2 + 3nm^2 + 5nm^2 = 9nm^2 \therefore \text{True for all } n, m \geq 1$$

Conclusion:

$$\therefore T(n, m)\Theta(nm^2)$$

1.11.2 Memoized

For COMMON-SUFFIX-LEN(i, j): (each of the nm subproblems is computed once, so any subsequent call/lookup takes $O(1)$)

$$R(i,j) = 1 \therefore R(i,j) \Theta(1)$$

For the overall time complexity: (table setup + loop work + constants)

$$T(n,m) = nm + nm(R(i,j)+3) + 5T(n,m) = nm + nm(1+3) + 5T(n,m) = nm + 4nm + 5T(n,m) = 5nm + 5$$

Proof for $\Theta(nm)$:

$$c_1 nm \leq 5nm + 5 \leq c_2 nm$$

Left Side: (pick $c_1 = 5$)

$$5nm \leq 5nm + 50 \leq 5 \therefore \text{True for all } n, m \geq 1$$

Right Side: (pick $c_2 = 10$)

$$5nm + 5 \leq 10nm5 \leq 5nm1 \leq nm \therefore \text{True for all } n, m \geq 1$$

Conclusion:

$$\therefore T(n,m) \Theta(nm)$$

1.11.3 Eager

For the overall time complexity: (table setup + nested loops + constants)

$$T(n,m) = nm + nm(4) + 5T(n,m) = 5nm + 5$$

Proof for $\Theta(nm)$:

$$c_1 nm \leq 5nm + 5 \leq c_2 nm$$

Left Side: (pick $c_1 = 5$)

$$5nm \leq 5nm + 50 \leq 5 \therefore \text{True for all } n, m \geq 1$$

Right Side: (pick $c_2 = 10$)

$$5nm + 5 \leq 10nm5 \leq 5nm1 \leq nm \therefore \text{True for all } n, m \geq 1$$

Conclusion:

$$\therefore T(n,m) \Theta(nm)$$

1.12 Benchmarking

[6]:
```
import timeit
import random
import string
import matplotlib.pyplot as plt
import numpy as np
```

```python
# Generate random strings of specified lengths
def generate_random_string(length: int) -> str:
    return ''.join(random.choices(string.ascii_lowercase, k=length))
def generate_same_string(length: int) -> str:
    return 'a' * length

# Define input sizes to test
# Note: Keep backtracking sizes small (up to n=150) because O(n * m * min(n,␣
 ↪m)) gets very slow!
sizes = [10, 25, 50, 75, 100, 150, 200, 300, 400, 500]

bench_results = {
    "Backtracking": [],
    "Memoized": [],
    "Eager": [],
    "Eager Optimized": []
}

# Run benchmark experiments
print(f"{'Size (n = m)':<15} | {'Backtracking (s)':<18} | {'Memoized (s)':<14}␣
 ↪| {'Eager (s)':<12} | {'Optimized (s)'}")
print("-" * 80)

for size in sizes:
    # Generate test strings of equal length for a controlled test
    a = generate_same_string(size)
    b = generate_same_string(size)

    # Benchmark Backtracking
    if size <= 500:
        t_backtrack = timeit.timeit(lambda: maximal_substring_backtracking(a,␣
 ↪b), number=5) / 5
    else:
        t_backtrack = float('nan') # Too slow for large inputs
    bench_results["Backtracking"].append(t_backtrack)

    # Benchmark Memoized
    t_memo = timeit.timeit(lambda: maximal_substring_memoized(a, b), number=20)␣
 ↪/ 20
    bench_results["Memoized"].append(t_memo)

    # Benchmark Eager
    t_eager = timeit.timeit(lambda: maximal_substring_eager(a, b), number=20) /␣
 ↪20
    bench_results["Eager"].append(t_eager)
```

```
    # Benchmark Eager Optimized
    t_opt = timeit.timeit(lambda: maximal_substring_eager_optimized(a, b),␣
 ↪number=20) / 20
    bench_results["Eager Optimized"].append(t_opt)

    bt_str = f"{t_backtrack:.5f}" if size <= 500 else "N/A (Timeout)"
    print(f"{size:<15} | {bt_str:<18} | {t_memo:<14.5f} | {t_eager:<12.5f} |␣
 ↪{t_opt:.5f}")

print("-" * 80)

# ------------------------------------------------------------
# Compute Log-Log Slopes (Empirical Time Complexity)
# ------------------------------------------------------------
slopes = {}
sizes_arr = np.array(sizes)

for name, times in bench_results.items():
    times_arr = np.array(times)
    # Filter out NaNs (e.g., from Backtracking timeout) and zeros/negative␣
 ↪values
    valid_mask = (~np.isnan(times_arr)) & (times_arr > 0)

    x_valid = sizes_arr[valid_mask]
    y_valid = times_arr[valid_mask]

    # Linear fit on log-log: log(y) = slope * log(x) + intercept
    slope, _ = np.polyfit(np.log(x_valid), np.log(y_valid), 1)
    slopes[name] = slope

print("\nEmpirical Log-Log Slope (Approximate Complexity when n = m):")
print(f"{'Algorithm':<20} | {'Log-Log Slope':<15} | {'Empirical Complexity':
 ↪<22}")
print("-" * 62)
for name, slope in slopes.items():
    print(f"{name:<20} | {slope:<15.2f} | O(n^{slope:.2f})")
print("-" * 62)

# ------------------------------------------------------------
# Plot with Slopes in Legend Labels
# ------------------------------------------------------------
plt.figure(figsize=(10, 6))

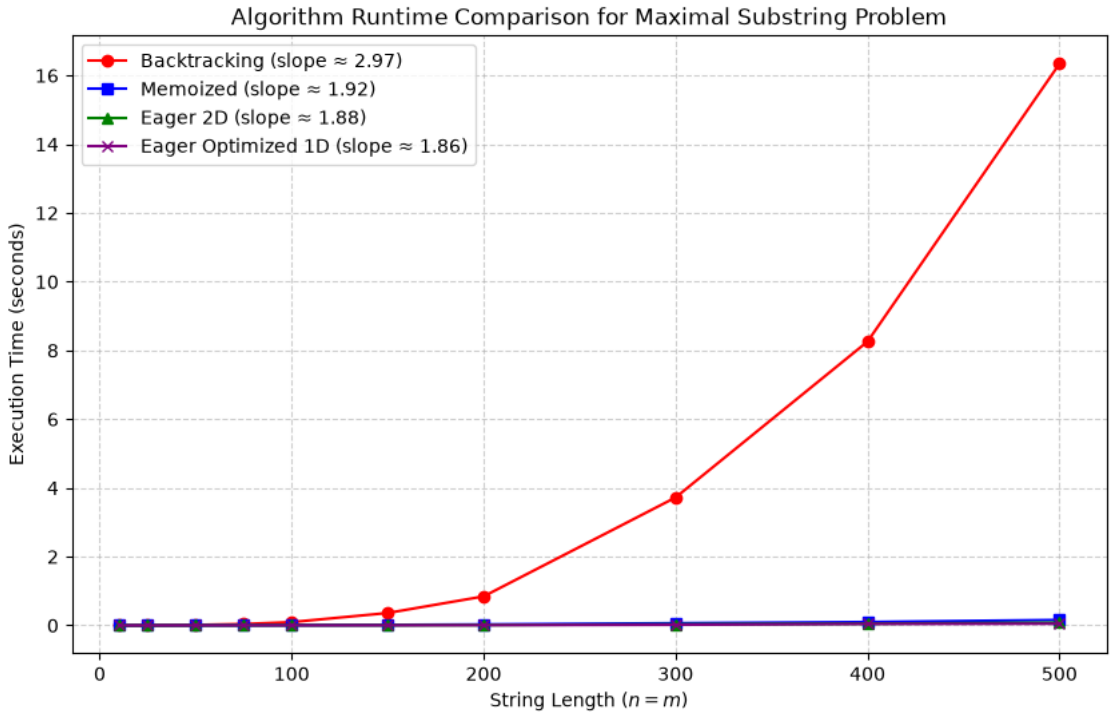
plt.plot(
    sizes, bench_results["Backtracking"],
    marker='o', color='red',
    label=f'Backtracking (slope  {slopes["Backtracking"]:.2f})'
```

```
)
plt.plot(
    sizes, bench_results["Memoized"],
    marker='s', color='blue',
    label=f'Memoized (slope  {slopes["Memoized"]:.2f})'
)
plt.plot(
    sizes, bench_results["Eager"],
    marker='^', color='green',
    label=f'Eager 2D (slope  {slopes["Eager"]:.2f})'
)
plt.plot(
    sizes, bench_results["Eager Optimized"],
    marker='x', color='purple',
    label=f'Eager Optimized 1D (slope  {slopes["Eager Optimized"]:.2f})'
)

plt.title('Algorithm Runtime Comparison for Maximal Substring Problem')
plt.xlabel('String Length ($n = m$)')
plt.ylabel('Execution Time (seconds)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.savefig('benchmark_results.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
Size (n = m)    | Backtracking (s)   | Memoized (s)   | Eager (s)    | Optimized
(s)
-------------------------------------------------------------------------------
10              | 0.00016            | 0.00008        | 0.00005      | 0.00003
25              | 0.00204            | 0.00056        | 0.00023      | 0.00019
50              | 0.01417            | 0.00221        | 0.00097      | 0.00079
75              | 0.03964            | 0.00380        | 0.00159      | 0.00137
100             | 0.09572            | 0.00793        | 0.00421      | 0.00188
150             | 0.36041            | 0.01508        | 0.00527      | 0.00339
200             | 0.84383            | 0.02772        | 0.01121      | 0.00667
300             | 3.72162            | 0.06429        | 0.03196      | 0.01799
400             | 8.25664            | 0.09763        | 0.05032      | 0.04300
500             | 16.33854           | 0.15780        | 0.07863      | 0.04922
-------------------------------------------------------------------------------

Empirical Log-Log Slope (Approximate Complexity when n = m):
Algorithm           | Log-Log Slope   | Empirical Complexity
-------------------------------------------------------------
Backtracking        | 2.97            | O(n^2.97)
Memoized            | 1.92            | O(n^1.92)
Eager               | 1.88            | O(n^1.88)
Eager Optimized     | 1.86            | O(n^1.86)
-------------------------------------------------------------
```



Our benchmarking shows that the memoized and eager methods are both approximately $O(n^2)$ which matches our theoretical result of $O(nm)$ when $n = m$, and the backtracking also appears to be $O(n^3)$ which matches our theoretical result of $O(n^2m)$ when $n = m$.